Software Architecture and Design Specification

Project: File Conversion Service (API) { CONVERTY }
Version: 1.0
Authors: Anivartha Upadhyaya, Ananya Joshi, Ayush Gowda
Date: 20-08-2025
Status: Final / Approved

Revision History

Version	Date	Author	Change Summary
1.0	20-08-2025	Converty Team	First complete architecture draft

---

Approvals

Role	Name	Signature/Date
Course Coordinator	Arvind Srinath K T	22-08-2025
Product Owner	Ananya Joshi	—
Dev Lead	Anivartha U	—
QA Lead	Ayush Gowda	—

---

1. Introduction

1.1 Purpose

This document specifies the **software architecture and design** for the **File Conversion Service (API)** named *Converty*. It defines system structure, components, data flow, UX considerations, and technical design decisions needed to build, test, and maintain the service.

1.2 Scope

The system supports:

- Uploading files in formats such as DOCX, PDF, CSV, JSON, Markdown
- Converting files to target formats (e.g., Markdown → PDF, CSV → JSON)
- Returning converted files via API or download
- Logging, monitoring, error handling, and performance constraints

1.3 Audience

Developers, QA Engineers, Architects, System Administrators, Instructors, and Maintenance Teams.

1.4 Definitions

API, CSV, PDF, JSON, MD, TLS, REST API, Conversion Engine, Logging Module.

---

2. Document Overview

2.1 How to Use This Document

This document provides architectural deliverables including:

- Component diagram
- Sequence diagram
- API definitions
- Technology stack
- Design rationale
- Traceability to SRS

2.2 Related Documents

- **SRS v1.0** (File Conversion Service)
- **STP v1.0**
- **Project Retrospective Report**
- **GitHub + Jira Boards**

---

3. Architecture

3.1 Goals & Constraints

Goals

- Accurate file conversion (≥99% correctness)
- High availability (99.5% uptime)
- Secure communication (TLS 1.2+)
- Scalable layered architecture
- Fast response (<3s for ≤10MB files)

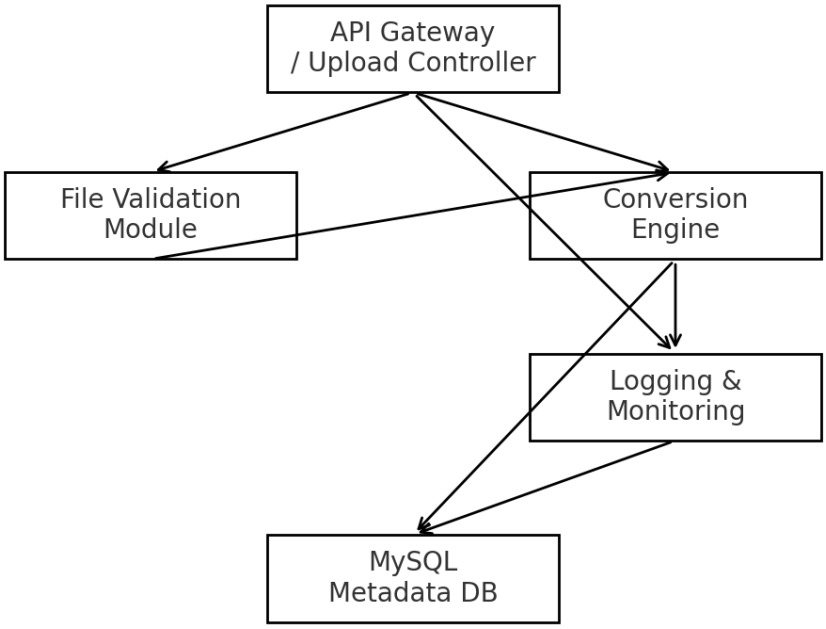
Constraints

- Only supported formats allowed
- File size limit (≤50MB)
- Pandoc & converter tools must be installed in environment
- REST API only

3.2 Stakeholders & Concerns

Stakeholder	Concerns
End Users	Easy upload/download, reliability, success rate
Developers	Modularity, testability, simple APIs
QA Engineers	Clear traceability, predictable behavior
Admins	Logging, monitoring, uptime, error visibility
Instructors	Documentation clarity
Security Auditors	Secure upload, sanitization, HTTPS

3.3 Component (UML) Diagram



3.4 Component Descriptions

1. API Gateway / Upload Controller

Handles incoming requests, routes to validation and conversion modules.

2. File Validation Module

Checks format, file size, corruption, MIME type, and security sanitization.

3. Conversion Engine

Uses Pandoc and other libraries to transform files between formats (CSV→JSON, MD→PDF, DOCX→TXT, etc).

4. Logging & Monitoring Module

Logs all conversion attempts with timestamps, input/output format, and status.

5. Storage / Metadata Database

MySQL DB storing conversion logs, metadata, errors, and system metrics.

6. Error Handler

Returns descriptive and user-friendly errors (unsupported format, corrupted file, etc).

3.5 Chosen Architecture Pattern & Rationale

Architecture Pattern: Layered Architecture

- **Presentation Layer:** API endpoints
- **Service Layer:** Validation + conversion logic
- **Data Layer:** Logging + metadata storage

Reason:
Simple, maintainable, clear separation of concerns, easy testing, ideal for student-scale API systems.

3.6 Technology Stack & Data Stores

- **Backend:** Node.js + Express + TypeScript
- **Conversion:** Pandoc + format-specific libraries
- **Database:** MySQL
- **Protocols:** REST over HTTPS
- **Tools:** Postman, JMeter, Swagger, ELK Stack

3.7 Risks & Mitigations

Risk	Mitigation
Large files slow conversions	Add size limit (50MB), optimize workers
Pandoc dependency failures	Pre-install & run environment checks
Invalid or malicious uploads	Strong validation + sanitization

Risk	Mitigation
Downtime	Cloud VM fallback environment
Git conflicts	Feature-branch workflow

3.8 Traceability to Requirements

Requirement ID	Architectural Element
FCS-F-001 File Upload	Upload Controller
FCS-F-010 Conversion	Conversion Engine
FCS-F-030 Error Handling	Error Handler
FCS-NF-001 Performance	Monitoring Module
FCS-NF-002 Uptime	Infrastructure Layer

3.9 Security Architecture

Security Threat (STRIDE)	Mitigation
Spoofing	API key/token validation
Tampering	TLS 1.2+, sanitized inputs
Repudiation	Detailed conversion logs
Information Disclosure	HTTPS, restricted file access
DoS	Rate limiting, file size limits
Elevation of Privilege	Role-based access for Admin

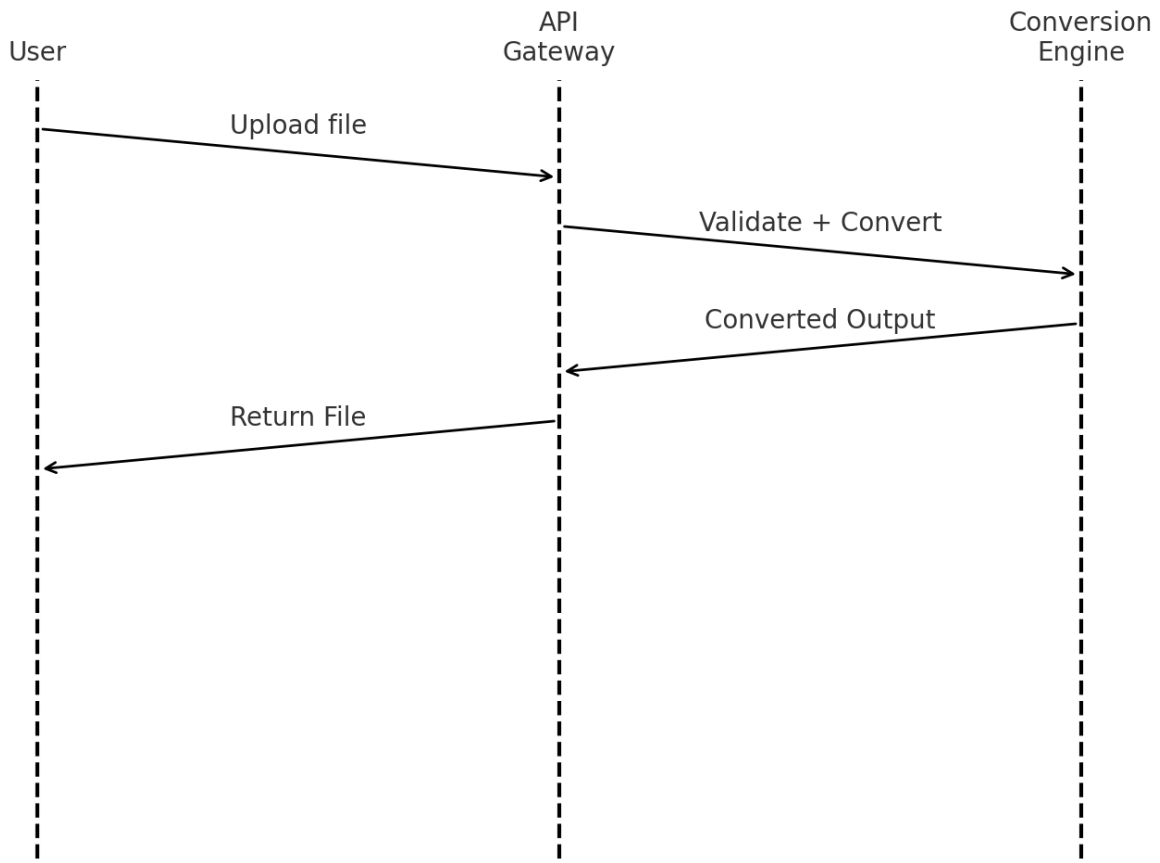
4. Design

4.1 Design Overview

The system is modular with clear division between API endpoints, validation logic, conversion services, logging, and error handling. Each component is independently testable.

4.2 UML Sequence Diagrams

Flow 1: File Upload → Validation → Conversion → Download



Steps:

1. User uploads a file via /upload.
2. Upload Controller validates format + size.
3. Validation Module approves the file.
4. Conversion Engine converts into target format using Pandoc.
5. Result is logged.
6. Converted file returned to user.

4.3 API Design

1. Upload File Endpoint

```
POST /convert
Request:
{
  "targetFormat": "pdf"
}
File: multipart/form-data

Response:
{
  "status": "success",
  "downloadUrl": ".../output.pdf"
}
```

2. Log Retrieval (Admin)

```
GET /logs
Response:
[
  { "input": "md", "output": "pdf", "time": "2.1s", "status": "success" }
]
```

4.4 Error Handling, Logging & Monitoring

- Clear error messages
- Reject unsupported formats or corrupted files
- No sensitive data stored
- Monitoring using ELK / Prometheus
- Logs retained

4.5 UX Design

- Simple web UI (optional) for drag-and-drop upload
- Shows conversion progress
- Clean confirmation/error messages
- Language and accessibility options

4.6 Open Issues & Next Steps

- Add more formats (e.g., XLSX, PPTX)
- Add queue system for large files
- Introduce CI/CD automated formatting, linting
- Add web-based dashboard for logs

---

5. Appendices

5.1 Glossary

API, Conversion Engine, CSV, JSON, Pandoc, Metadata, TLS, REST.

5.2 References

IEEE 42010, Pandoc Docs, Express Documentation, OWASP File Upload Guidelines.

5.3 Tools

VS Code, Postman, JMeter, Swagger UI, GitHub, Jira.